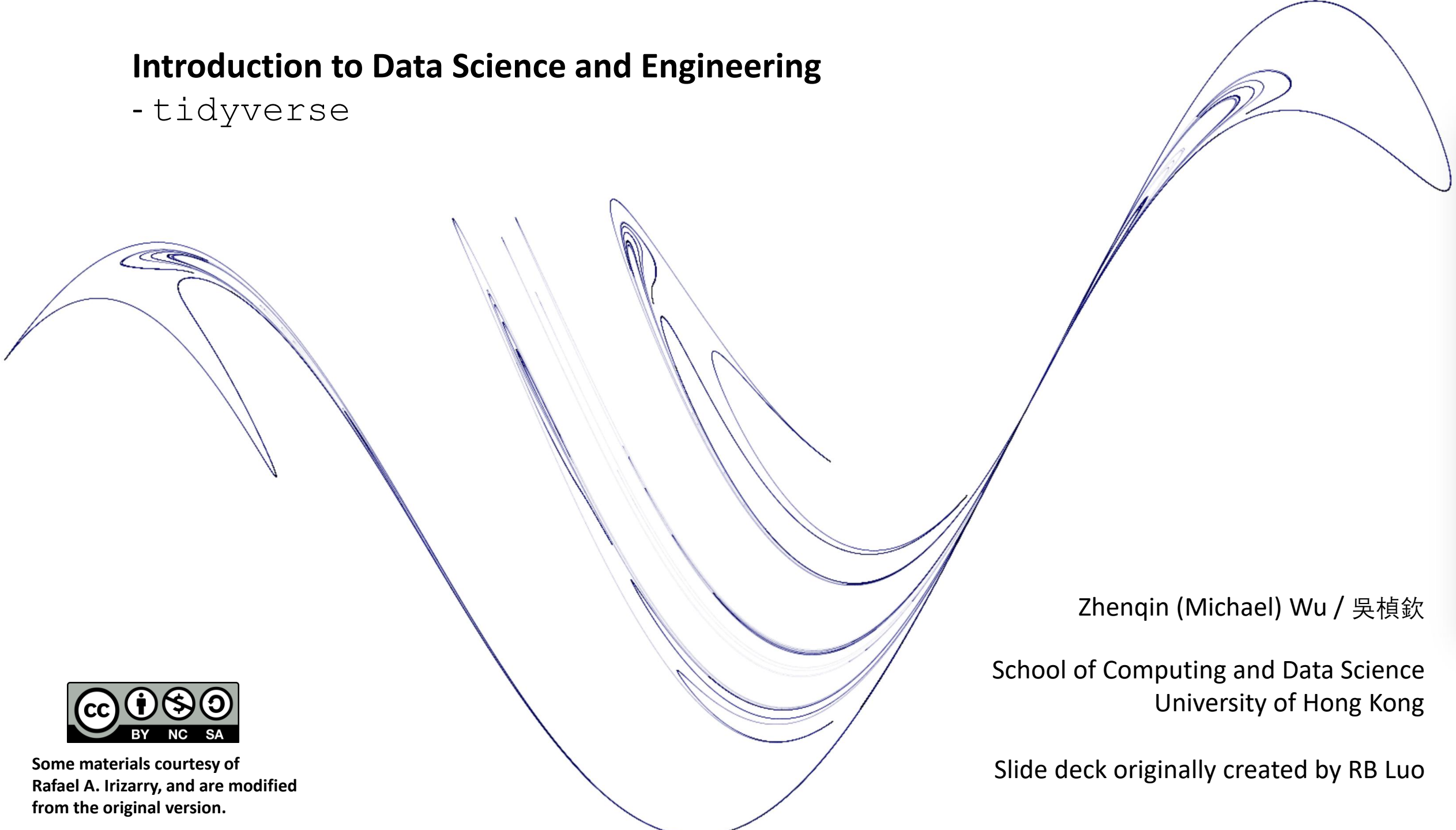
Introduction to Data Science and Engineering

- tidyverse



Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo

Some materials courtesy of Rafael A. Irizarry, and are modified from the original version.

Feb, 2023

TA Write R code to analyze the trend of COVID-19 in the U.S.

Here's some sample R code that could be used to analyze the trend of COVID-19 cases in the United States:

scss Copy code

```
# Load required libraries
library(tidyverse)
library(readxl)

# Load data
covid_data <- read_excel("https://raw.githubusercontent.com/nytimes/covid-19

# Filter for U.S. data only
us_covid_data <- covid_data %>%
  filter(country == "US")

# Group data by date and sum cases
us_covid_data_grouped <- us_covid_data %>%
  group_by(date) %>%
  summarize(total_cases = sum(cases))

# Plot trend of total cases over time
ggplot(us_covid_data_grouped, aes(x = date, y = total_cases)) +
  geom_line() +
  ggtitle("Trend of COVID-19 Cases in the United States") +
  xlab("Date") +
  ylab("Total Cases")
```

This code first loads the required libraries, then loads COVID-19 data from the New York Times GitHub repository, filters the data to include only U.S. data, groups the data by date and sums the cases, and finally plots the trend of total cases over time.

Note that this code is just a sample to get you started and you may need to adjust it based on your specific data and analysis needs.

Jan, 2024

You
Write R code to analyze the trend of COVID-19 in the U.S.

ChatGPT
Certainly! To analyze the trend of COVID-19 in the U.S., you can use R and leverage libraries like `ggplot2` for data visualization and `dplyr` for data manipulation. Below is an example code using publicly available COVID-19 data from the "COVID-19 Data Repository by the Center for Systems Science and Engineering (CSSE) at Johns Hopkins University."

R Copy code

```
# Install and load required packages
install.packages(c("tidyverse", "lubridate"))
library(tidyverse)
library(lubridate)

# Download COVID-19 data from Johns Hopkins University
url <- "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/maste
covid_data <- read.csv(url, stringsAsFactors = FALSE)

# Filter data for the United States
us_covid_data <- covid_data %>% filter(Country.Region == "US")

# Convert data to long format for time series analysis
us_covid_data_long <- us_covid_data %>%
  select(-Province.State, -Country.Region, -Lat, -Long) %>%
  pivot_longer(cols = starts_with("X"), names_to = "Date", values_to =
  mutate(Date = mdy(Date))

# Plot the trend of COVID-19 cases in the U.S.
ggplot(us_covid_data_long, aes(x = Date, y = Cases, group = 1)) +
  geom_line(color = "blue") +
  labs(title = "COVID-19 Trend in the U.S.",
       x = "Date",
       y = "Number of Cases") +
  theme_minimal()
```

Make sure to install the required packages (`tidyverse` and `lubridate`) before running the code. You can install them using `install.packages(c("tidyverse", "lubridate"))`.

August, 2025

ChatGPT 5 Share

Write R code to analyze the trend of COVID-19 in the U.S.

已思考 17s

Here's a clean, end-to-end R script that pulls public data, builds core trend metrics, and plots the U.S. COVID-19 trajectory. It uses **Our World in Data** for national trends (simple + stable) and optionally the **CDC Socrata API** for state-level detail.

r Copy Edit

```
# ---- Setup ----
packages <- c("tidyverse", "lubridate", "scales", "zoo")
to_install <- setdiff(packages, rownames(installed.packages()))
if (length(to_install)) install.packages(to_install, repos = "https://cloud.r-pro
invisible(lapply(packages, library, character.only = TRUE))

# ---- Load national data (Our World in Data) ----
# Source: https://ourworldindata.org/coronavirus
owid_url <- "https://covid.ourworldindata.org/data/owid-covid-data.csv"
us <- readr::read_csv(owid_url, show_col_types = FALSE) |>
  filter(location == "United States") |>
  select(date,
         population,
         new_cases,
         new_cases_smoothed,
         new_deaths,
         new_deaths_smoothed,
         new_tests,
         new_tests_smoothed,
         hosp_patients,                    # in hospital (snapshot), not all
         weekly_hosp_admissions,           # per OWID (may be sparse)
         icu_patients,
         total_cases, total_deaths) |>
  mutate(
    cases_7dma = new_cases_smoothed,
    deaths_7dma = new_deaths_smoothed,
    tests_7dma  = new_tests_smoothed,
    cases_per_100k_7dma  = (cases_7dma / population) * 1e5,
    deaths_per_100k_7dma = (deaths_7dma / population) * 1e5,
    week = floor_date(date, "week", week_start = 1)
  )
```

In this lecture

- `tidyverse` and tidy data frames
- Manipulate data frames
- `purr` and `map`

In this lecture

- **`tidyverse` and tidy data frames**
- Manipulate data frames
- `purr` and `map`

The `tidyverse`

In R basics, we learned **manipulating vectors** by reordering and subsetting them through indexing. However, once we start more advanced analyses, the preferred unit for data storage is not the vector but the data frame.

In this chapter we learn to work directly with data frames (and we will be using data frames in all following lectures).

Load the library

Ya, simple.

```
> library(tidyverse)
── Attaching packages ─────────────────────────────────────── tidyverse 1.3.2 ──
✔ ggplot2 3.4.0      ✔ purrr   1.0.0
✔ tibble  3.1.8      ✔ dplyr   1.0.10
✔ tidyr   1.2.1      ✔ stringr 1.5.0
✔ readr   2.1.3      ✔ forcats 0.5.2
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
```

Tidy data

A data table is **tidy** if:

- each row represents **ONE** observation;
- each column represent a different variable available for observations

```
> library(dslabs)
> data(murders)
> murders
                  state abb        region population total
1               Alabama  AL         South    4779736   135
2                Alaska  AK          West     710231    19
3               Arizona  AZ          West    6392017   232
4              Arkansas  AR         South    2915918    93
5            California  CA          West   37253956  1257
6              Colorado  CO          West    5029196    65
7           Connecticut  CT     Northeast    3574097    97
8              Delaware  DE         South     897934    38
9  District of Columbia  DC         South     601723    99
10              Florida  FL         South   19687653   669
```

Tidy data

Which one is tidy?

```
> data(co2)
> co2
        Jan    Feb    Mar    Apr    May    Jun    Jul    Aug    Sep    Oct    Nov    Dec
1959 315.42 316.31 316.50 317.56 318.13 318.00 316.39 314.65 313.68 313.18 314.66 315.43
1960 316.27 316.81 317.42 318.87 319.87 319.43 318.01 315.74 314.00 313.68 314.84 316.03
1961 316.73 317.54 318.38 319.31 320.42 319.61 318.42 316.63 314.83 315.16 315.94 316.85
1962 317.78 318.40 319.53 320.42 320.85 320.45 319.45 317.25 316.11 315.27 316.53 317.53
1963 318.58 318.92 319.70 321.22 322.08 321.31 319.58 317.61 316.05 315.83 316.91 318.20
1964 319.41 320.07 320.74 321.40 322.06 321.73 320.27 318.54 316.54 316.71 317.53 318.55
1965 319.27 320.28 320.73 321.97 322.00 321.71 321.05 318.71 317.66 317.14 318.70 319.25
1966 320.46 321.43 322.23 323.54 323.91 323.59 322.24 320.20 318.48 317.94 319.63 320.87
1967 322.17 322.34 322.88 324.25 324.83 323.93 322.38 320.76 319.10 319.24 320.56 321.80
1968 322.40 322.99 323.73 324.86 325.40 325.20 323.98 321.95 320.18 320.09 321.16 322.74
```

```
> co2_df <- data.frame(
+     Year = floor(time(co2)),
+     Month = cycle(co2),
+     CO2 = as.numeric(co2)
+ )
> co2_df
    Year Month    CO2
1   1959     1 315.42
2   1959     2 316.31
3   1959     3 316.50
4   1959     4 317.56
5   1959     5 318.13
6   1959     6 318.00
7   1959     7 316.39
8   1959     8 314.65
9   1959     9 313.68
10  1959    10 313.18
11  1959    11 314.66
12  1959    12 315.43
13  1960     1 316.27
14  1960     2 316.81
```

In this lecture

- `tidyverse` and tidy data frames
- **Manipulate data frames**
- `purr` and `map`

Adding a column with `mutate`

Analyze the `murders` dataset: the total count of murders is not meaningful, how to calculate and save the per capita gun murders?

The function `mutate` takes:

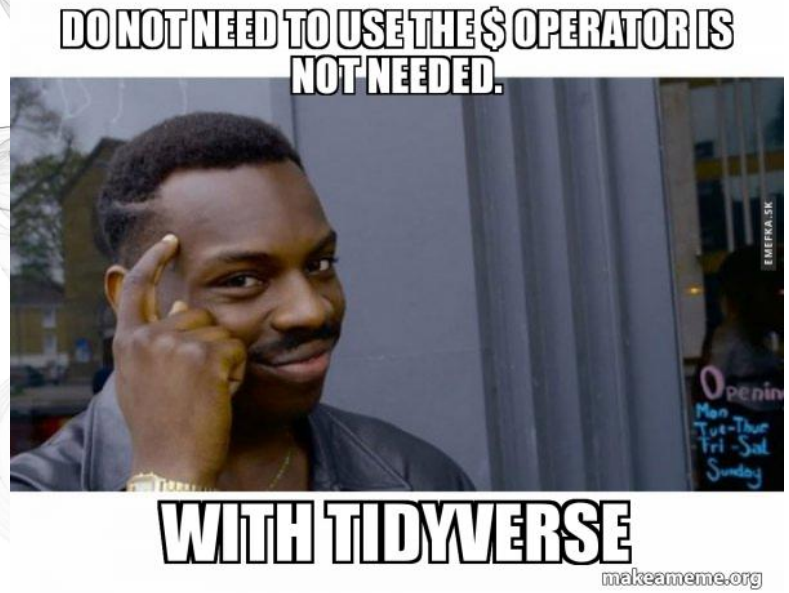
1. The data frame as the first argument;
2. Name and values of the variable as the second argument.

```
> library(dslabs)
> data("murders")
> murders <- mutate(murders, rate = total / population * 100000)
> head(murders)
       state abb region population total     rate
1    Alabama  AL  South    4779736   135 2.824424
2     Alaska  AK   West     710231    19 2.675186
3    Arizona  AZ   West    6392017   232 3.629527
4   Arkansas  AR  South    2915918    93 3.189390
5 California  CA   West   37253956  1257 3.374138
6   Colorado  CO   West    5029196    65 1.292453
```

Notice that here we used `total` and `population` inside the function, which are objects that are not defined in our workspace. But why don't we get an error?

This is one of the main feature of `dplyr/tidyverse`. Functions know to look for columns of the data frame. In the call to `mutate` above, `total` is equivalent to `murders$total`.

Similarly, `rate` will be defined as a new column of the target data frame




```
> library(dslabs)
> data("murders")
> murders <- mutate(murders, rate = total / population * 100000)
> head(murders)
       state abb region population total     rate
1    Alabama  AL  South    4779736   135 2.824424
2     Alaska  AK   West     710231    19 2.675186
3    Arizona  AZ   West    6392017   232 3.629527
4   Arkansas  AR  South    2915918    93 3.189390
5 California  CA   West   37253956  1257 3.374138
6   Colorado  CO   West    5029196    65 1.292453
```

Subsetting rows with `filter`

Now suppose that we want to filter the data table to only show the entries for which the murder rate is lower than 0.71.

To do this we use the function `filter`, which takes:

1. The data frame as the first argument;
2. The conditional statement as the second argument.

```
> filter(murders, rate <= 0.71)
          state abb        region population total      rate
1        Hawaii  HI          West    1360301     7 0.5145920
2          Iowa  IA North Central    3046355    21 0.6893484
3 New Hampshire  NH     Northeast    1316470     5 0.3798036
4  North Dakota  ND North Central     672591     4 0.5947151
5       Vermont  VT     Northeast     625741     2 0.3196211
```

Selecting columns with `select`

To only show columns of interest, we can use the function `select`.

The code below extracts three columns, assigns them to a new object, and further filters it.

The function `select` takes:

1. The data frame as the first argument;
2. Column names of interest as following arguments.

Note that the data frame is always the first argument

```
> new_table <- select(murders, state, region, rate)
> filter(new_table, rate <= 0.71)
          state        region      rate
1        Hawaii          West 0.5145920
2          Iowa North Central 0.6893484
3 New Hampshire     Northeast 0.3798036
4  North Dakota North Central 0.5947151
5       Vermont     Northeast 0.3196211
```

Piping

In R we can perform a series of operations: add a new column, select some columns, filter observations, etc., by sending the results of one function to the subsequent one using the pipe operator

|> %>%

Piping almost always use the output of the left expression as the FIRST argument to the right expression.

```
> murders |> select(state, region, rate) |> filter(rate <= 0.71)
          state        region      rate
1        Hawaii          West 0.5145920
2          Iowa North Central 0.6893484
3 New Hampshire     Northeast 0.3798036
4  North Dakota North Central 0.5947151
5       Vermont     Northeast 0.3196211
```

Piping

Result of the left expression is sent to the right expression as its first argument

```
> 16 |> sqrt()
[1] 4
> 16 |> sqrt() |> log2()
[1] 2
```

is equivalent to

```
> x <- 16
> x <- sqrt(x)
> x
[1] 4
> x <- log2(x)
> x
[1] 2
```

Other arguments can be defined as usual

```
> 16 |> sqrt() |> log(base = 2)
[1] 2
```

Piping is also extensively used in Unix/Linux shell:
https://www.geeksforgeeks.org/piping-in-unix-or-linux/

Piping - placeholder

```
> log(8, base = 2)
[1] 3
> 2 |> log(8, base = _)
[1] 3
```

Piping |> allows seamless data frame manipulation operations. No need to name intermediate objects.

What if we want to pass results as the second/third/etc. argument to the right expression?

- Use placeholder operator _

Summarizing data with `summarize`

An important part of exploratory data analysis is summarizing data: e.g., average, standard deviation.

The function `summarize` provides a handy way to compute summary. The example below shows calculating summary statistics on the `heights` dataset:

```
> library(dplyr)
> library(dslabs)
> data(heights)
> s <- heights |> filter(sex == "Female") |> summarize(average = mean
(height), standard_deviation = sd(height))
> s
   average standard_deviation
1 64.93942           3.760656
```

This code snippet first subsets the data frame to keep only females, then produces a new summary statistics table with average and standard deviation.

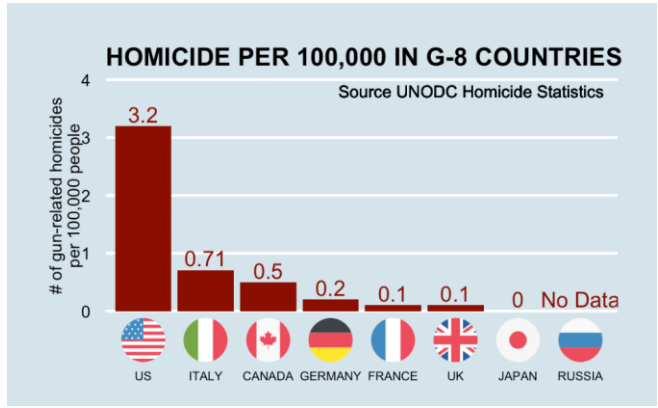
Difference from `mutate`:

- `summarize` is supposed to return only one row

Summarizing data with `summarize`

For another example, let's compute the average gun murder rate in the United States. Starting from:

```
> murders <- murders |> mutate(rate = total/population*100000)
```



Is this correct?

```
> murders |> summarize(avg_rate = mean(rate))
  avg_rate
1 2.779125
```

What's the difference?

```
> murders |> summarize(avg_rate = sum(total) / sum(population) * 100000)
  avg_rate
1 3.034555
```

Multiple summaries

Suppose we want three summaries from the same variable such as the minimum, median, and maximum heights. The quantile function: `quantile(x, c(0, 0.5, 1))` returns the min (0th percentile), the median (50th percentile), and max (100th percentile).
We can use it with summarize like this

```
> heights |> filter(sex == 'Female') |> summarize(min_median_max = quantile(height, c(0, 0.5, 1)))
  min_median_max
1       51.00000
2       64.98031
3       79.00000
Warning message:
Returning more (or less) than 1 row per `summarise()` group was deprecated in dplyr
1.1.0.
i Please use `reframe()` instead.
i When switching from `summarise()` to `reframe()`, remember that `reframe()` always
  returns an ungrouped data frame and adjust accordingly.
```

Note the warning

It is more recommended to return one row, but with multiple columns. To do that:

```
> min_median_max <- function(x){
+     qs <- quantile(x, c(0, 0.5, 1))
+     data.frame(min = qs[1], median = qs[2], max = qs[3])
+ }
> heights |>
+ filter(sex == "Female") |>
+ summarize(min_median_max(height))
  min   median max
1  51 64.98031  79
```

group_by then summarize

```
> heights |> group_by(sex)
# A tibble: 1,050 × 2
# Groups:   sex [2]
   sex    height
   <fct>   <dbl>
 1 Male       75
 2 Male       70
 3 Male       68
 4 Male       74
 5 Male       61
 6 Female     65
 7 Female     66
 8 Female     62
 9 Female     66
10 Male       67
# … with 1,040 more rows
# ℹ Use `print(n = ...)` to see more rows
```

A common operation in data exploration is to first stratify data into groups and then perform the same operations (e.g., summary statistics) on them separately. For example, compute average and standard deviation of heights for male and female separately.

The `group_by` function helps us do this: it transforms a data frame into **a grouped data frame**, and `dplyr` functions, in particular `summarize`, will behave separately for different groups.

```
> data("heights")
> heights |>
+   group_by(sex) |>
+   summarize(n = n(), avg = mean(height), sd = sd(height))
# A tibble: 2 × 4
  sex        n   avg    sd
  <fct>  <int> <dbl> <dbl>
1 Female   238  64.9  3.76
2 Male     812  69.3  3.61
```

Conceptually, you can think of a grouped data frame as many tables, with the same columns but non-overlapping rows. Use `n()` to get the count of different grouups

group_by then summarize

```
> murders |>
+   group_by(region) |>
+   summarize(min_median_max(rate), n=n())
# A tibble: 4 × 5
  region          min median   max     n
  <fct>         <dbl>  <dbl> <dbl> <int>
1 Northeast     0.320   1.80  3.60     9
2 South         1.46    3.40 16.5     17
3 North Central 0.595   1.97  5.36    12
4 West          0.515   1.29  3.63    13
```

For another example, let's compute the minimum, median, and maximum murder rate in the four regions of US.

Sorting data frames

Similar to `order`, the function `arrange` can be used to order entries of a data frame:

```
> murders |>
+ arrange(population) |>
+ head()
                  state abb        region population total       rate
1               Wyoming  WY          West     563626     5  0.8871131
2 District of Columbia  DC         South     601723    99 16.4527532
3               Vermont  VT     Northeast     625741     2  0.3196211
4          North Dakota  ND North Central     672591     4  0.5947151
5                Alaska  AK          West     710231    19  2.6751860
6          South Dakota  SD North Central     814180     8  0.9825837
```

```
> murders[order(murders$population),] |> head()
                  state abb        region population total       rate
51              Wyoming  WY          West     563626     5  0.8871131
9  District of Columbia  DC         South     601723    99 16.4527532
46              Vermont  VT     Northeast     625741     2  0.3196211
35         North Dakota  ND North Central     672591     4  0.5947151
2                Alaska  AK          West     710231    19  2.6751860
42         South Dakota  SD North Central     814180     8  0.9825837
```

Sorting data frames

Note that the default behavior is to order in ascending order.
Use the function `desc` for descending order:

```
> murders |>
+ arrange(desc(rate)) |>
+ head()
                  state abb        region population total      rate
1 District of Columbia  DC         South     601723    99 16.452753
2             Louisiana  LA         South    4533372   351  7.742581
3              Missouri  MO North Central    5988927   321  5.359892
4              Maryland  MD         South    5773552   293  5.074866
5        South Carolina  SC         South    4625364   207  4.475323
6              Delaware  DE         South     897934    38  4.231937
```

Nested sorting

Sorting can be by multiple columns. Here we order by region, then within region we order by murder rate descendingly

```
> murders |>
+ arrange(region, desc(rate)) |>
+ head()
          state abb    region population total     rate
1  Pennsylvania  PA Northeast   12702379   457 3.597751
2    New Jersey  NJ Northeast    8791894   246 2.798032
3   Connecticut  CT Northeast    3574097    97 2.713972
4      New York  NY Northeast   19378102   517 2.667960
5 Massachusetts  MA Northeast    6547629   118 1.802179
6  Rhode Island  RI Northeast    1052567    16 1.520093
```

Sorting data frames, `top_n`

Sometimes we just care about the extremes. The function `top_n` helps to show only the top n records according to a specified column (not sorted).

```
> murders |> top_n(5, rate)
                  state abb        region population total      rate
1 District of Columbia  DC         South     601723    99 16.452753
2             Louisiana  LA         South    4533372   351  7.742581
3              Maryland  MD         South    5773552   293  5.074866
4              Missouri  MO North Central    5988927   321  5.359892
5        South Carolina  SC         South    4625364   207  4.475323
```

tidyverse conditionals (between)

```
> x >= a & x <= b
```

To check if the elements of a vector `x` are between `a` and `b`:

```
> between(x, a, b)
```

Use the `between` function to perform the same operation; `x`, `a`, `b` can all be vectors.

```
> between(rnorm(100), seq(-2, -1, length.out=100), rep(1, times=100))
  [1]  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE
 [17] FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE FALSE  TRUE  TRUE FALSE
```

In this lecture

- `tidyverse` and tidy data frames
- Manipulate data frames
- **`purr` and `map`**

The purrr package

```
> compute_s_n <- function(n){
+     x <- 1:n
+     sum(x)
+ }
> n <- 1:25
> s_n <- sapply(n, compute_s_n)
```

In the R basics, we learned `sapply` function, which enabled us to apply the same function to each element of a vector. We constructed a function and used `sapply` to compute the sum of the first n integers.

This type of operation: applying the same operations to all elements of a list/vector, is quite common in data analysis.

The `purrr` package provides similar functions that:

- Work better with other `tidyverse` functions.
- Strict control over output data types.

The `purrr` package

```
> library(purrr)
> s_n <- map(n, compute_s_n)
> class(s_n)
[1] "list"
```

The function `map`, which is very similar to `sapply`, always returns **a list**.

```
> s_n <- map_dbl(n, compute_s_n)
> class(s_n)
[1] "numeric"
```

The function `map_dbl` always returns **a vector of numeric values**.

```
> compute_s_n <- function(n){
+     x <- 1:n
+     data.frame(sum = sum(x))
+ }
> s_n <- map_df(n, compute_s_n)
```

The function `map_df` always returns **a data frame**. However, the inner function needs to return a data frame or something that can be converted to a data frame (e.g., list with names).

map series

Function	Input	Output	Key Features	Comparison to `apply` Series
`map()`	List or atomic vector	List	Iterates over elements of a list/vector and applies a function, always returns a list.	Similar to `lapply()`, but more consistent and flexible.
`map_dbl()`, `map_lgl()`, `map_int()`, `map_chr()`	List or atomic vector	Vector of atomic data types: (`logical`, `integer`, `numeric`, `character`)	Applies a function and ensures the result is coerced to the specific data type.	Similar to `sapply()`
`map_df()`	List or atomic vector	Data frame	Applies a function and binds the results row-wise into a data frame.	Comparable to `apply()` in the row bind mode
`pmap()`	List of lists/vectors	List	Applies a function to multiple arguments from multiple lists/vectors (`...`).	Similar to `mapply()

In this lecture

- `tidyverse` and tidy data frames
- Manipulate data frames
  - `mutate, filter, select, |>`
  - `summarize, group_by`
  - `arrange, desc, top_n, between`
- `purr` and `map`
  - `map, map_dbl, map_df`